1. Contexto

Bancolombia busca fortalecer su equipo de Inteligencia Artificial con ingenieros capaces de diseñar, construir y desplegar soluciones de IA generativa en producción. Esta prueba técnica evalúa su capacidad para construir un sistema RAG (Retrieval-Augmented Generation) end-to-end, desde la adquisición de datos hasta la interacción con el usuario final a través de un agente conversacional.

> Importante: Esta prueba no busca una solución perfecta ni un producto terminado. Busca entender cómo piensa, diseña, implementa y documenta soluciones técnicas. Valoramos las decisiones de arquitectura tanto como el código funcional.

2. Objetivo

Construir un asistente virtual del Grupo Bancolombia que responda preguntas sobre productos, servicios y contenido publicado en la sección de personas del sitio web, utilizando una arquitectura RAG con un agente conversacional accesible mediante una interfaz de chat.

3. Alcance de la Solución

La solución debe cubrir las siguientes capas de principio a fin:

3.1. Adquisición de datos (Web Scraping / Crawling)

Implementar un proceso automatizado que:

- Recorra https://www.bancolombia.com/personas y descubra los links internos de artículos, productos y páginas informativas.
- Descargue el contenido textual de cada página encontrada.
- Almacene los datos crudos de forma estructurada con metadatos: URL de origen, título, fecha de extracción, categoría.

> *No se requiere agotar todo el sitio. Se espera un mínimo de 50 páginas/artículos procesados. El candidato debe documentar sus decisiones sobre profundidad de crawling, manejo de contenido dinámico (JavaScript rendering) y estrategia de respeto a robots.txt.*

3.2. Procesamiento y limpieza de datos

Implementar un pipeline de limpieza que:

- Elimine HTML, navegación, footers, banners y contenido no informativo.
- Extraiga el contenido relevante de cada página: texto principal, títulos, subtítulos.

- Chunking: aplique una estrategia de segmentación del texto adecuada para recuperación semántica. El candidato debe justificar la estrategia elegida: tamaño del chunk, overlap, método de segmentación.
- Genere un dataset limpio y listo para la etapa de embedding.

3.3. Generación de embeddings y almacenamiento vectorial

Implementar el proceso de:

- Generación de embeddings a partir del texto limpio utilizando un modelo de embeddings (abierto o comercial).
- Almacenamiento en una base de datos vectorial: Pinecone, ChromaDB, Weaviate, Qdrant, OpenSearch, pgvector u otra.
- Indexación con metadatos que permitan filtrado posterior: URL de origen, categoría, etc.

> *El candidato debe justificar: la elección del modelo de embeddings, la dimensionalidad, la base vectorial seleccionada y la estrategia de indexación.*

3.4. Retrieval y Generación (RAG) — Servidor MCP

El componente de recuperación y generación debe implementarse como un servidor MCP (Model Context Protocol). MCP es un estándar abierto que permite exponer capacidades (tools, resources, prompts) de forma que cualquier cliente compatible pueda consumirlas. Al implementar el RAG como servidor MCP, la base de conocimiento se convierte en una capacidad reutilizable, desacoplada del agente y del frontend.

Tools obligatorias del servidor MCP

Tool	Descripción
search_knowledge_base	Recibe una consulta en lenguaje natural, ejecuta búsqueda semántica contra la base vectorial y retorna los documentos más relevantes con metadatos (URL, título, score de relevancia).
get_article_by_url	Recibe una URL del sitio de Bancolombia y retorna el contenido completo del artículo indexado correspondiente.
list_categories	Retorna las categorías disponibles en la base de conocimiento, si el candidato implementó clasificación por categoría durante el scraping.

Resource obligatorio del servidor MCP

Resource URI	Descripción
knowledge-base://stats	Expone estadísticas de la base de conocimiento: número de documentos indexados, categorías disponibles, fecha de última actualización, modelo de embeddings utilizado.

Requisitos técnicos del servidor MCP

- SDK oficial: @modelcontextprotocol/sdk (TypeScript/Node) o mcp con FastMCP (Python).
- Transporte: soporte obligatorio para stdio. SSE o Streamable HTTP es deseable pero opcional.
- Validación de parámetros en cada tool con descripciones claras — estas descripciones son las que el LLM usará para decidir cuándo invocar cada herramienta.
- Manejo de errores: base vectorial no disponible, query sin resultados, timeouts, etc.

Flujo RAG dentro de las tools: búsqueda semántica → reranking opcional → retorno de documentos con fuentes (URLs). La construcción del prompt y la invocación del LLM pueden ocurrir en el servidor MCP o en el agente cliente — el candidato debe justificar su decisión.

Modelos aceptados: cualquier LLM accesible por API o ejecutable localmente: Claude, GPT, Gemini, Llama, Mistral, etc. Se puede usar el tier gratuito de cualquier proveedor.

Referencia: https://modelcontextprotocol.io · SDKs en https://github.com/modelcontextprotocol

3.5. Agente conversacional (Cliente MCP)

El agente debe actuar como cliente MCP, conectándose al servidor implementado en el paso anterior para acceder a la base de conocimiento. El agente debe:

- Conectarse al servidor MCP e invocar las tools expuestas según la intención del usuario.
- Mantener contexto de la conversación: historial de mensajes.
- Decidir cuándo consultar la base de conocimiento y cuándo responder directamente (saludos, clarificaciones).
- Manejar preguntas fuera de alcance (temas no relacionados con Bancolombia).
- Citar las fuentes (URLs) retornadas por el servidor MCP en cada respuesta.
- Seguridad para el consumo del agente.
- Manejo de las diferentes tipologías de memoria que podría usar el agente (corto, mediano y largo plazo).

Framework para el agente: libre elección. LangChain, LangGraph, CrewAI, Autogen, N8N, SDK de Anthropic con tool use, o implementación propia. Lo importante es que la conexión con la base de conocimiento se realice a través del protocolo MCP.

3.6. Frontend — Interfaz de chat

Implementar una interfaz de usuario tipo chat que:

- Permita al usuario escribir preguntas y recibir respuestas del agente.
- Muestre el historial de la conversación.
- Presente las fuentes (URLs) utilizadas para generar cada respuesta.
- Sea funcional y usable. No se evalúa diseño gráfico avanzado, sí claridad y usabilidad.

Tecnologías aceptadas: React, Next.js, Vue, Svelte, Streamlit, Gradio o cualquier framework de su preferencia.

4. Requisitos No Funcionales

Requisito	Descripción
Arquitectura limpia	El código debe seguir principios de arquitectura limpia o hexagonal: separación de capas, inversión de dependencias, bajo acoplamiento. Se evalúa la estructura del proyecto, no solo que funcione.
Diseño de la solución	Entregar un diagrama de componentes que muestre la arquitectura general: servicios, bases de datos, APIs, flujos de datos. Herramienta libre: draw.io, Excalidraw, Mermaid, etc.
Documentación	README.md profesional con: descripción del proyecto, diagrama de arquitectura, instrucciones de instalación, decisiones técnicas justificadas y limitaciones conocidas.
CI/CD	Pipeline CI que ejecute linting y/o tests al hacer push. Pipeline CD implementado o documentado. Se acepta GitHub Actions, GitLab CI, Bitbucket Pipelines u otra herramienta.
Repositorio Git	Código en repositorio público (GitHub, GitLab o Bitbucket). El historial de commits debe reflejar desarrollo incremental, no un solo commit con todo.
Contenerización	La solución debe poder ejecutarse con docker-compose up (o equivalente). Incluir Dockerfile(s) para cada servicio.
Variables de entorno	API keys y configuraciones sensibles manejadas con variables de entorno. Incluir .env.example con variables requeridas sin valores reales.

5. Criterios de Evaluación

Criterio	Peso	Qué se evalúa
Diseño y arquitectura	25 %	Calidad del diagrama de componentes, separación de responsabilidades, decisiones de arquitectura justificadas, escalabilidad del diseño.
Implementación RAG + MCP	25 %	Pipeline completo funcionando: scraping → limpieza → embeddings → retrieval → generación. Servidor MCP con tools, resources y validación. Calidad de las respuestas, citación de fuentes, manejo de errores.
Agente (cliente MCP) y frontend	20 %	Agente conectado al servidor MCP, invocación correcta de tools, manejo de contexto conversacional, interfaz de chat usable, experiencia de usuario.
Calidad del código	15 %	Código limpio y modular. Arquitectura limpia implementada (no solo mencionada). Manejo de errores. Testing unitario en componentes críticos.
DevOps y documentación	15 %	CI/CD funcionando, Docker configurado correctamente, README profesional, instrucciones claras de ejecución, decisiones técnicas documentadas.

6. Entregables

1. Repositorio Git público con todo el código fuente, Dockerfiles y configuración de CI/CD.
2. README.md con documentación completa (ver sección 4).
3. Diagrama de componentes de la solución (incluido en el README o como archivo separado).
4. Video demo de 3–5 minutos mostrando la solución funcionando: el chat en acción con al menos 5 preguntas diferentes sobre productos y servicios de Bancolombia, mostrando cómo las respuestas incluyen las fuentes. Puede ser una grabación de pantalla con narración o con texto explicativo en pantalla.

7. Restricciones y Aclaraciones

Punto	Detalle
Lenguaje	Libre. Python, TypeScript/JavaScript, Java, Go u otro.
Costos	La solución no debe requerir costos elevados. Use tiers gratuitos de APIs de LLMs, bases vectoriales y servicios cloud. Si usa un modelo local, documente los requisitos de hardware.
Prohibido	Entregar una solución que solo funcione con datos hardcodeados o respuestas preprogramadas. El sistema debe hacer retrieval real contra la base vectorial.
Propiedad intelectual	El código desarrollado es de su autoría. Bancolombia no retendrá ni utilizará su código sin su consentimiento.
IA como herramienta	Puede usar Copilot, Claude, ChatGPT, etc. para asistir el desarrollo. Sin embargo, debe poder explicar cada decisión técnica en la entrevista posterior. El código entregado debe ser comprendido en su totalidad por el candidato.

8. Cronograma

Hito	Plazo
Envío de la prueba al candidato	Martes 06 de abril del 2026, 12:00 pm
Fecha límite de entrega	12 días calendario desde el envío (Viernes 17 de abril del 2026, 11:59 pm)
Revisión técnica por el equipo	4 días hábiles después de la entrega
Entrevista técnica (presentación)	Por agendar tras revisión

La entrega se realiza compartiendo el link del repositorio público por correo a la dirección que se indicará en la invitación.

9. Ejemplo de Interacción Esperada

Para dar claridad sobre el resultado esperado, este es un ejemplo ilustrativo de cómo debería funcionar el sistema:

Usuario	¿Qué opciones de crédito de vivienda tiene Bancolombia?
Agente	Bancolombia ofrece varias opciones de crédito de vivienda: 1. Crédito de vivienda en pesos: para adquisición de vivienda nueva o usada, con plazos de hasta 20 años. 2. Crédito de vivienda en UVR: con cuotas que se ajustan según la inflación. 3. Leasing habitacional: alternativa donde el banco adquiere el inmueble y usted paga un canon mensual. *Fuentes consultadas:* • https://www.bancolombia.com/personas/creditos/vivienda/credito-vivienda

Nota: Este ejemplo es ilustrativo. Las respuestas reales dependerán del contenido que el candidato logre extraer del sitio web.

10. Preguntas Frecuentes

¿Puedo usar servicios cloud pagos?
Sí, pero la solución debe poder evaluarse sin que el evaluador incurra en costos. Incluya instrucciones para usar alternativas gratuitas o proporcione credenciales temporales si es necesario.

¿Qué profundidad de crawling se espera?

Mínimo 50 páginas procesadas. No es necesario agotar toda la sección /personas. Demuestre que el pipeline funciona y que más páginas se podrían agregar fácilmente.

¿Nunca he trabajado con MCP, puedo usar otra cosa para el RAG?

No. El servidor MCP es un requisito obligatorio de la prueba. MCP es un estándar abierto con documentación clara y SDKs oficiales en Python, TypeScript y JAVA. La curva de aprendizaje es parte de lo que se evalúa: capacidad de adoptar tecnologías nuevas, leer documentación y construir una implementación funcional.

¿Qué transporte debo usar para el servidor MCP?

El transporte stdio es obligatorio (es el más simple y permite probar localmente). Si adicionalmente implementa SSE o Streamable HTTP para consumo remoto, será valorado positivamente pero no es requisito.

¿Puedo usar un LLM local en vez de una API?

Sí. Documente los requisitos de hardware y cómo ejecutarlo. Considere que el evaluador puede no tener GPU disponible: incluya una alternativa con API si es posible.

¿Se evalúa el diseño visual del frontend?

No se espera un diseño visual elaborado. Se evalúa que sea funcional, claro y que permita una conversación fluida. Streamlit o Gradio son completamente aceptables.

¿Qué pasa si el web scraping falla en algunas páginas?

Es esperado. Documente qué páginas no pudieron procesarse y por qué. Se evalúa el manejo de errores y la resiliencia del pipeline, no que logre el 100 % de las páginas.

¿Puedo entregar antes de los 12 días?

Sí. La entrega anticipada no otorga puntos extra ni penaliza, pero sí demuestra capacidad de ejecución.

Proceso de Selección · Bancolombia · Área Tecnología e IA TI · 6 de abril de 2026